

Problemi su Alberi

All'interno dei sistemi distribuiti, il problema principale è rappresentato dalla simmetria

Shout con molti initiator

Questo perché in una situazione di perfetta simmetria i nodi non riescono a capire quale azione eseguire

Gli Alberi hanno una struttura asimmetrica di natura pertanto non è necessaria l'UI per romperla

Saturation

La saturation è una tecnica utilizzata per risolvere vari problemi sugli alberi andando a selezionare 1 o 2 nodi

i quali in seguito eseguiranno dei calcoli indispensabili per l'esecuzione di varie task

Ad esempio il calcolo del minimo

e del centro dell'albero

La Saturation è una tecnica che si basa su 3 fasi:

a) Activation phase:

Wake-up

Partendo dagli Initiator tutti i nodi vengono svegliati

b) Saturation phase:

Partendo dalle foglie, una coppia di nodi vicini tra loro viene scelta

c) Resolution phase:

Partendo dalla coppia scelta, viene inviato a tutti i nodi un msg di notifica per la terminazione locale dei nodi

Per descrivere più nello specifico queste fasi avremo bisogno di definire per bene il modello

Assunzioni

Lavoreremo sotto le restrizioni di:

• $R \rightsquigarrow$ Standard $\rightsquigarrow$ BL, CN, IR

• Knowledge of Topology

Più nello specifico sanno se sono nodi interni o foglie

I nodi sanno di essere in un albero

Stati possibili

- Available $\rightarrow$ I nodi devono ancora ricevere il primo msg
- Active $\rightarrow$ il nodo aspetta una risposta da tutti i suoi vicini
- Processing $\rightarrow$ il nodo si trova in uno stato d'attesa in cui aspetta un msg dal padre
- Saturated $\rightarrow$ stato in cui si trova il nodo eletto

$\rightarrow$ dopo esser stati svegliati nell' Activation phase

Configurazioni iniziali e finali

Analizzeremo ora i possibili configurazioni iniziali e finali per quanto riguarda la Saturation phase

- Configurazione iniziale:

$$\forall x \in V: \text{status}(x) = \text{Available}$$

- Configurazione finale:

$$\exists ! \{x, y\} \text{ con } x, y \in V \text{ t.c. } (x, y) \in E \text{ AND } \text{status}(x) = \text{status}(y) = \text{Saturated}$$



Idea dietro al protocollo

Dopo l'Activation phase tutti i nodi si trovano nello stato Available

A questo punto le Soglie inizieranno la Saturation phase

- a) Ogni Soglia invia un msg M al suo unico vicino e diventa Processing

$\rightarrow$ il padre

- b) Quando un nodo interno riceve M da tutti i vicini tranne uno
invia M all'unico vicino da cui non lo abbia ricevuto e diventa Processing

$\rightarrow$ il padre

- c) Quando un nodo nello stato Processing riceve M dal padre allora questo diventa Saturated

$\rightarrow$ che dimostreremo in seguito

Nel momento in cui i due nodi divengono Saturated verrà iniziata la Resolution phase

Protocol Full Saturation

- Status: $S = \{\text{Available}, \text{Active}, \text{Processing}, \text{Saturated}\}$
 $S_{init} = \{\text{Available}\}$
- Restrictions: $R \cup T \rightsquigarrow$ topology knowledge

Available

- Spontaneously

send(Activate) to $N(x)$

Neighbour = $N(x)$

if Neighbour = ϵ then

$M = ("Saturation")$

parent $\leftarrow$ Neighbour

send(M) to parent

become Processing

else

become Active

Se e' Soglia

- Receiving(Activate)

send(Activate) to $N(x) - \{\text{Sender}\}$

Neighbour = $N(x)$

if Neighbour = ϵ then

$M = ("Saturation")$

parent $\leftarrow$ Neighbour

send(M) to parent

become Processing

else

become Active

Se e' Soglia

Active

- Receiving(M)

$Neighbour = Neighbour - \{Sender\}$

if $Neighbour = \emptyset$ then

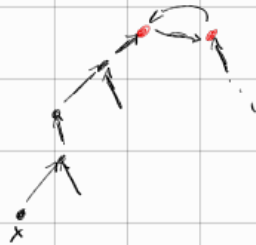
$M = ("Saturation")$

$parent \leftarrow Neighbour$

send(M) to parent

become Processing

x ha ricevuto M da tutti i vicini tranne uno



Processing

- Receiving(M)

become Saturated

Correttezza

Esattamente 2 nodi Processing diventeranno Saturated e sono vicini

Dim

- Dimostriamo innanzitutto che almeno 2 nodi sono Saturated

Per dimostrarlo prendiamo un qualsiasi nodo $x \in V$

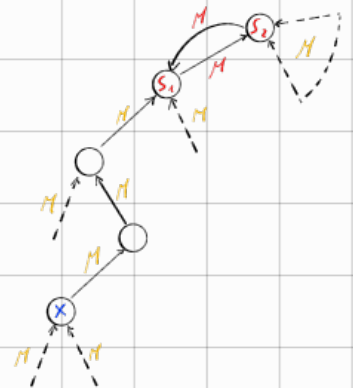
Consideriamo ora il cammino composto dagli archi attraversati da M

Continuando a risalire con il cammino prima o poi si incontrerà un nodo saturo s_1

s_1 è diventato Saturo dopo aver ricevuto M da s_2

Visto che s_2 invia M ad s_1 , s_2 si trova nello stato Processing

Per cui anche s_2 diventa Saturo dopo aver ricevuto M da s_1



- Adesso ci manca solo da dimostrare che i nodi saturi sono esattamente 2 nodi adiacenti

Assumiamo per assurdo che ci siano più di due nodi saturi

Allora $\exists$ due nodi saturi x, y t.c. $d(x, y) \geq 2$

Consideriamo allora un nodo z sul cammino da x verso y

z non può inviare M sia verso x che y quindi uno dei due non può essere Saturo



Oss

La scelta della coppia di nodi saturati e' strettamente dipendente dai communication delays e pertanto non e' possibile prevedere quale coppia sia

Analisi Msg Complexity

Analizziamo ora il # di msg scambiati in ogni fase imponendo la presenza di K initiator

a) Activation stage:

$$M[\text{Wake-up}] = n + K - 2$$

b) Saturation stage:

Su ogni arco passa esattamente 1 msg M grazie che sull'arco che connette i 2 nodi saturi su cui ne passano 2

$$M[\text{Saturation}] = n - 1 + 1 = n$$

c) Resolution stage:

I due nodi saturi danno partire dei broadcast sui loro rispettivi sotto alberi

$$M[\text{Resolution}] = n - 2$$

Activation + Saturation

Pertanto la msg complexity del full Saturation e'

$$M[\text{Full Saturation}] = M[\text{Wake-up}] + M[\text{Saturation}] = n + K - 2 + n = 2n + K - 2$$

Non stiamo contando la Resolution phase

Applicazioni della Saturation

Un'importante tipo di problemi sono quelli di Distributed Function Evaluation

Ovvero di task basate sul calcolare funzioni i cui argomenti si trovano nei nodi del sistema distribuito

Fra questi problemi ce ne sono molti che possono essere risolti mediante la Saturation che però devono essere

un semi gruppo, ovvero devono avere le proprietà di:

- Associatività:

$$F(a, b, c) = F(F(a, b), c)$$

- Commutatività:

$$F(a, b, c) = F(b, a, c)$$

All'interno di questa categoria di funzioni troviamo ad esempio:

- Minimum/Maximum
- Sum/Product
- Logical Function

Minimum Finding

Come già accennato in precedenza la funzione Min è un semigrupp e come tale può essere risolta tramite Saturation

Senza andare troppo nei dettagli il problema presenta le seguenti configurazioni iniziali Sinali

- Configurazione iniziale:

Al proprio interno ogni nodo $x \in V$ presenta un valore $value(x)$

- Configurazione Sinali:

Alla Sinali ogni nodo deve sapere se sia o no quello con valore minimo

Si può notare come in alberi radicati si può risolvere la task senza utilizzare Saturation in quanto basta eseguire:

- 1) Broadcast dalla root fino alle foglie
- 2) Convergecast dalle foglie verso la root in modo da trasportarne i valori
- 3) La root calcola il minimo fra i valori consegnati
- 4) Broadcast dalla root in modo da notificare tutti i nodi del minimo trovato

Per risolvere la task su reti più generali invece bisogna usare la Saturation in modo che:

- a) Il nodo della Root viene sostituito dai due nodi Saturati
- b) Le Soglie mandano il msg M contenente il loro valore $val(x)$
- c) Quando i nodi interni ricevono tutti i msg M dai loro figli, ne calcolano il minimo e lo inviano al padre
- d) I due nodi Saturati conosceranno il minimo e lo comunicheranno tramite Broadcast alla rete

Questa soluzione ovviamente ha la Msg Complexity della Saturation ovvero

$$M_{sg}(\text{Min Finding}) = 4n - 4 = O(n)$$

Naive: Broadcast della richiesta da ogni nodo

Convergenza dalle soglie per collezionare i valori

$$M_{sg} = O(n^2)$$

Calcolo dell'eccentricità

Definisco l'eccentricità di una rete come:

$$e(x) = \max \{d(x, y) \mid \forall x, y \in V\}$$

Il calcolo di quest'ultimo è un'altra task risolvibile tramite Saturation

L'idea alla base è la seguente:

- 1) Calcolare l'eccentricità dei due nodi Saturati usando il msg M
- 2) Propagare a partire dai nodi Saturati le info necessarie per calcolare l'eccentricità di ogni nodo

Per calcolare l'eccentricità dei nodi Saturati ci viene in aiuto il seguente Thm:

Thm

Siano $s_1, s_2 \in V$ due nodi adiacenti t.c. $s_1 \in T_1$ e T_1, T_2 i rispettivi sottoalberi

Allora

$$e(s_1) = \max \{h(T_1), 1 + h(T_2)\}$$

Per le info contenute all'interno di M al termine della Saturation Phase ogni nodo conosce l'altezza del suo sottoalbero

Per quanto riguarda l'informazione mancante questa viene propagata a partire dai nodi saturati

Si può notare come anche in questo caso la msg complexity sia esattamente quella della Saturation ovvero:

$$M(\text{Eccentricities}) \leq 4n - 4$$

Center Sinding

Un'altra task risolvibile tramite Saturation e' il Center Sinding, ovvero al termine del protocollo ogni nodo deve essere in grado di capire se sia o no il centro della rete

Visto che e' possibile definire il centro come il nodo con l'eccentricita' minore, una possibile soluzione potrebbe essere:

- 1) Si esegue la Saturation per il calcolo dell'eccentricita'
- 2) Saturation per il calcolo del minimo

Questa idea ha un costo di $M(*) \leq 6n-6$

Un'idea piu' intelligente fa uso di 3 proprieta' del centro ovvero:

- 1) In un albero il centro e' unico oppure sono due nodi vicini tra loro
- 2) I centri si trovano sul diametro path $\leadsto$ cammino piu' lungo dell'albero
- 3) Un nodo x e' un centro $\Leftrightarrow d_1[x] - d_2[x] \leq 1$

$\hookrightarrow$ dove d_1 e d_2 indicano le due distanze maggiori

Grazie alla terza prop. possiamo risolvere la task semplicemente applicando il protocollo per il calcolo dell'eccentricita' e quindi con msg complexity

$$M(*) \leq 4n-4$$